hw2实验报告

1.Tensor类的实现

1.Tensor的核心成员

```
std::vector<int> shape_; 保存 Tensor 每个维度的大小；
size_t numel_; 保存 Tensor 中元素的总数量；
Device device_; 表示当前 Tensor 数据位于 CPU 还是 GPU；
std::shared_ptr<float> data_; 保存实际数据，并负责数据内存的生命周期管理。
```

2.内存管理使用

使用shared_ptr智能管理内存的生命周期

- CPU:使用自定义deleter使用$delete[]$自动释放内存
- GPU:使用cuda的相关API管理内存

3.实现cpu与gpu相互之间的传输

- cpu()将数据从gpu转移到cpu上，如果本来就在cpu就返回本身
- gpu()将数据从cpu转移到gpu上，如果本来就在gpu就返回本身

2.Activation

已知

$$\mathrm{ReLU}(x) = \max(x, 0)$$

$$\frac{\partial L}{\partial x} = \begin{cases} \dfrac{\partial L}{\partial y}, & x > 0, \\ 0, & x \leq 0. \end{cases}$$

$$\mathrm{Sigmoid}(x) = \sigma(x) = \frac{1}{1 + \exp(-x)}$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} y(1 - y)$$

辅助函数

先实现了在gpu上面的relu和sigmoid函数前向传播和反向传播的函数分别为```

```
__global__ void relu_forward_kernel(const float* x,float* y,int n)

__global__ void relu_backward_kernel(const float* grad_output,const float*
x,float* grad_input,int n)

__global__ void sigmoid_forward_kernel(const float* x,float* y,int n)

__global__ void sigmoid_forward_kernel(const float* x,float* y,int n)
```

Relu和Sigmoid

已知两个函数的数学公式

- cpu:直接使用for循环遍历data进行操作即可
- gpu:利用上面的辅助函数进行并行运算

3.验证正确性

直接写一个程序通过比较简单的Tensor验证正确性，具体的编译之后的输出结果如下

```
D:\classes\ai编\hw2>.\test.exe
=== Test 1: Tensor Device Management ===
Original CPU Data: -2 -1 0 1 2 3
Transferred Back to CPU :-2 -1 0 1 2 3

=== Test 2: ReLU CPU & GPU Execution ===
ReLU Forward Output (GPU Result): 0 0 0 1 2 3
ReLU Backward Grad (GPU Result): 0 0 0 1 1 1

=== Test 3: Sigmoid CPU & GPU Execution ===
Sigmoid Forward Output (GPU Result): 0.119203 0.268941 0.5 0.731059 0.880797 0.952574
Sigmoid Backward Grad (GPU Result): 0.104994 0.196612 0.25 0.196612 0.104994 0.0451767
```